

Secure authentication system writeup

1. Overview

Before I created the system, I first decided on the core functionality of the project. This helps break the huge task down into smaller, more manageable tasks, making the project seem much easier and more doable. I decided that the program must be able to do the following:

- Authenticate user logins
- Prevent attacks such as SQL injection, brute force attacks and timing attacks
- Ensure that sensitive user data remains secure in the event of a data leak
- Be able to access a database containing user information via SQL code
- Ensure logged on users can only access their own data and nobody else's
- Once users have logged on, they should be able to create, access and delete their own data

Once I had decided on the key functions of the project, I chose to think about what database software to use. I ended up choosing SQLite for the following reasons:

1. It does not require a server and requires minimal configuration
2. For a small amount of data, SQLite is very efficient
3. Files can easily be backed up, moved, and shared.
4. It allows me to focus on secure coding principles rather than setting up complex databases

SQLite was chosen because it is a simple to use, low-concurrency database software focused on data handling instead of large-scale infrastructure. This was important for this project because its purpose is to simply show how you would create a secure authentication system. By using SQLite, the project also became very portable due to the database being contained in a single, small file.

2. Threat Modelling

Before I attempt to identify what threats may pose an issue to the system, I need to first outline what data needs to be protected. This data will likely consist of usernames, password hashes, salts, the database file, login attempt histories, encrypted user notes, and the key to decrypt the notes. Next, I need to identify who the likely attackers will be to try and gain access to this data. This will likely consist of existing users attempting to gain access to another user's data, automated bots attempting user credentials, and external attackers who wish to gain access to private user information.

Now that I have outlined what I am trying to protect and who may want to gain access to it, I can outline the ways that attackers may attempt to gain access. With the attackers and data in consideration, these are the realistic attack methods that were identified:

1. SQL injection via credential inputs
2. Brute force attacks to guess user passwords
3. Timing-based attacks during password authentication
4. Data leaks leading to the exposure of private information

These threats were selected based upon their realistic ability to succeed in a real-world authentication system and potential impact they could have if they succeed. If any of these attacks were to succeed, sensitive information such as salts, password hashes, encrypted notes and login attempts could be exposed, leading to a compromise of the system's confidentiality.

3. Attack Mitigation

SQL injection

```
sql = "SELECT * FROM data WHERE user_id = ?"
cursor.execute(sql, (ID,))
```

To prevent SQL injection, I used placeholders when passing variables into SQL code rather than string concatenation. This prevents attackers from injecting malicious SQL code through input fields by treating user input solely as data.

Brute-force attacks

```
sql = "SELECT timestamp FROM attempts WHERE username = ? AND result = 'DENIED' ORDER BY timestamp DESC LIMIT ?"
databaseHandler.cursor.execute(sql, (username, maxAttempts))
rows = databaseHandler.cursor.fetchall()

if len(rows) < maxAttempts:
    return False
else:
    latestAttempt = datetime.strptime(rows[0][0], "%Y-%m-%d %H:%M:%S.%f")
    lockoutUntil = latestAttempt + timedelta(seconds=lockoutSecs)
    return datetime.now() < lockoutUntil
```

The system tracks failed login attempts via the “attempts” table and checks which attempts were marked as “DENIED”. If there have been more than five failed attempts, then the system will lock the user out for 5 minutes. This greatly reduces the impact of Brute-force attacks as repeated attempts will be limited.

Timing-based attacks

```
def checkPassword(salt, hashed_pass, inputted_password):  
    newHashed_password = hashlib.pbkdf2_hmac(  
        "sha256",  
        inputted_password.encode(),  
        salt,  
        100_420  
    )  
    return hashed_pass == newHashed_password
```

The entered password is hashed using the same salt and hashing algorithm as the hashed password stored in the “auth” table and is then compared against it. Because the entered password is hashed before comparison, any small differences in authentication time are insignificant, as the system is comparing hashed values, and the hashing process contributes to most of the overall time.

Data leaks

```
def encryptNote(note):  
    return cipher.encrypt(note.encode())  
  
def decryptNote(note):  
    return cipher.decrypt(note).decode()
```

Before being stored in the database, all user notes are encrypted using Fernet symmetric encryption before being stored in the database. This ensures that if the database is leaked or an attacker has gained unauthorized access to the database, sensitive user data remains protected as long as the attacker does not have the encryption key.

Access control

```
def viewNotes(ID):  
    sql = "SELECT * FROM data WHERE user_id = ?"  
    cursor.execute(sql, (ID,))
```

Whenever a user decides to view their notes, the data access query uses the user's unique ID as a primary key to filter all notes to find solely their own. This prevents authenticated users from seeing other users' data even if they have entered the correct credentials to view their own notes.

Logging and monitoring

```
def loginAttemptLog(username, result, reason):  
    vals = (username, datetime.now(), result, reason)  
    sql = "INSERT INTO attempts(username, timestamp, result, reason) VALUES(?,?,?,?)"  
    cursor.execute(sql, vals)  
    connection.commit()
```

Whenever a user attempts to log in to the system, the username used, timestamp of the login attempts and whether the attempt was successful or not is all written to the "attempts" table in the database. This allows for general surveillance of login attempts and to prevent brute-force attacks as users will be prevented from logging in if they have had too many failed attempts.

4. Limitations & Potential Improvements

Whilst the program does display secure programming concepts, due to the scale of the program and purpose of the project, there are several limitations preventing the system from being able to function at a real-world level. These limitations help identify potential areas for improvement for future iterations of the system if it was to be made bigger or turned into a real-world system.

Key Management

Limitation:

Currently, the encryption key is being stored in the same file as the database and the source code for the program. This could lead to a potential attack because if an attacker was to gain access to the source code or server, they would likely be able to decrypt all user stored notes.

Potential Improvement:

In a real-world system, encryption keys should be stored using secure software like a key vault or environment variables. This would highly reduce the risk of the key being exposed if the database is compromised.

Brute-Force Prevention

Limitation:

The lockout mechanism is currently based on a fixed amount of failed login attempts which are tracked using the username as a base to lock someone out. This approach could easily be exploited by attackers via account lockout abuse to cause denial of service by creating bots to repeatedly guess someone's password, permanently locking them out.

Potential Improvement:

Firstly, this could be improved by locking out users based on a combination of their IP address and username used instead of their username. This will reduce the risk of attackers from creating bots to repeatedly guess incorrect passwords, locking users out of their own account. Next, I could increase lockout durations and use CAPTCHA to prevent attackers from making bots which use denial of service attacks.

Database Scalability

Limitation:

This database system uses SQLite, and uses code to add, view, and delete attributes of the database. Whilst this is suitable for a small-scale project like this one, which focuses more on secure programming fundamentals than creating a database to be used with real data. SQLite uses a locking system which could cause performance issues if there are a large quantity of users attempting to write to the database at the same time. It is also important to note that SQLite is not designed to serve multiple clients and be hosted on a server; this is due to it not handling high-concurrency client-server workloads well.

Potential Improvement:

To make this system suitable for a large-scale system for real-world use, I would consider migrating to a client-server database like MySQL or PostgreSQL. This will improve support for simultaneous users and concurrency handling.

Logging and Monitoring

Limitation:

Firstly, when suspicious activity is happening on the database such as repeated failed attempts at a login, there is no alert system to notify a system admin to the issue. Because of this, administrators are forced to manually read through logs to find potential attacks, allowing malicious activity to continue longer than necessary. Secondly, whilst the database does log all login attempts to the server, it will not log any attempts to create, read or delete notes. Without logging these actions, malicious behaviour like bots repeatedly sending requests could go undetected. This makes it harder to respond efficiently to application-level denial-of-service attacks. Finally, if an attacker gains access to the database, they could modify and delete logs to hide malicious activity because logs are stored in the same database as application data.

Potential Improvement:

Firstly, I would implement a real-time alerting system which notifies administrators when suspicious behaviour is occurring. This could include activity such as repeated failed login attempts and unusual request patterns. Secondly, I would modify my code to introduce logging for all actions such as creating, viewing, and deleting notes. This will improve visibility of application behaviour. Finally, I would protect logs against modification by making them read-only where possible.

5. Conclusion & Reflection

What The Project Demonstrates

Firstly, this project fulfils its goals of demonstrating secure programming principles. This was achieved by analysing the potential threats to the system and implementing security-focused code to prevent these threats. Examples of this are using placeholders during SQL code, hashing passwords and encrypting notes. With all of this considered, this project demonstrates the ability to design a system with security as the focus from the beginning.

Key Learning Points & Reflection

This project has taught me many important principles for designing a secure system. I gained a greater understanding of how attacks like SQL injection, timing-based attacks, brute-force attacks and data leaks can impact authentication systems. This allowed me to apply my theoretical knowledge to practical scenarios such as this project. This project also improved my ability to

identify weaknesses in a system, as when designing my own I went through the system several times to find weaknesses. An example of this is timing-based attacks which were discovered when I began to question whether checking passwords by simply comparing strings in Python would lead to any weaknesses in the system.

Future Direction

If developed further, the likely first step would be to improve security surrounding key storage. This would be done by using software such as a key vault or environment variables. Next, if the database was to be used on a larger scale, then I would likely switch from SQLite to a software like MySQL or PostgreSQL. This will allow the database to support more concurrent requests, making it more suitable to be hosted on a server to be accessed over a network. This project has also increased my awareness of the weaknesses of this system; therefore, I would improve security by implementing features such as better rate limiting and log monitoring systems.

Designing and analysing this system has further reinforced my interest in cyber security by implementing theoretical knowledge in a practical environment, showing how security-focused measures are applied and how they may be exploited. In conclusion, this project has allowed me to develop a security-focused mindset, greater reasoning ability surrounding threats and mitigations, and greater problem-solving skills.